

Advanced Machine Learning

Lecture 9: LSTM & GRU

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

10 December 2024



Outline

Repetition

- Recurrent Neural Networks
- Cells & layers
- Elman network
- Backpropagation & Training

RNN: properties & variants

- Problems w/ vanilla RNN
- Long-term dependencies
- LSTM & variants
- BPTT for LSTM

Outline

Repetition

Recurrent Neural Networks

Cells & layers

Elman network

Backpropagation & Training

RNN: properties & variants

Problems w/ vanilla RNN

Long-term dependencies

LSTM & variants

BPTT for LSTM

Motivation

All network structures we considered so far are constrained to a **fixed input** and **fixed output size**.

This poses problems for **sequences of data**, e.g., **text**, **speech**, **video**:

- ▶ the input may be **sequences of arbitrary lengths** (e.g., text analysis, video frames)
- ▶ the output may be **sequences of arbitrary lengths** (translation of spoken or written language)

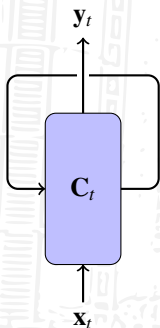
Moreover, the **amount of computation** in the network structures thus far does **not depend on the input**. Here, an interesting application would be to **process data** in a **sequential manner** by some policy depending on the input.

The **solution** to these problems are so-called **Recurrent Neural Networks (RNN)**.

Definition of an RNN cell

Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t$, $t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t$, $t = 0, 1, 2, \dots$ that should be learned by a neural network.

An **RNN cell** for this problem looks as follows:



It has a **cell state** $\mathbf{C}_t$, $t = 0, 1, \dots$ and an **additional output** that is **used** in the **next step as additional input**.

In **most applications**, the cell state is **internally completely described by**:

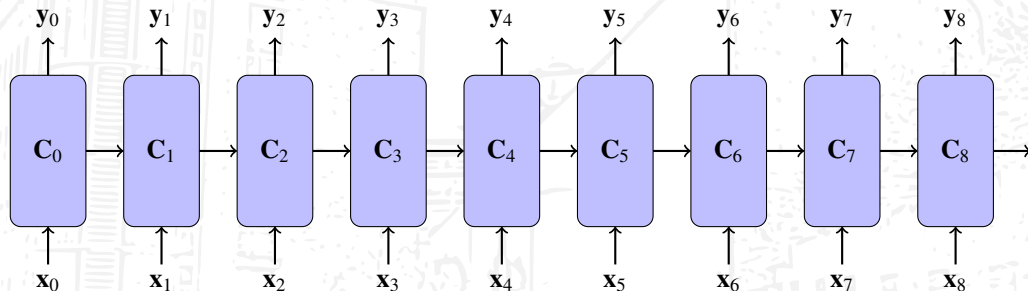
- ▶ **fixed data**, e.g., some fixed matrices, and
- ▶ the **variable update vector** that is handed to the next iteration, the **hidden state**.

Thus, RNN can **store information** (“a program”).

Unfolding an RNN cell

To better understand how RNN work, RNN cells are frequently **unfolded**. This will be crucial when deriving the **training of RNN**.

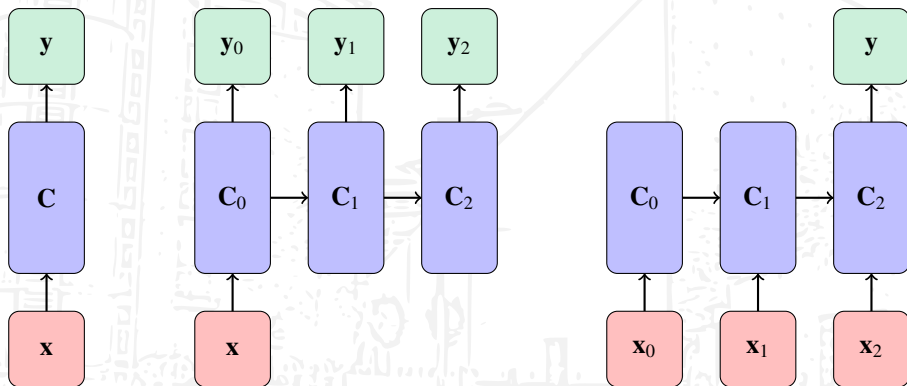
Example: an RNN cell **unfolded for 9 steps**



Thus, an RNN can be considered as a neural network that **shares weights over time**.

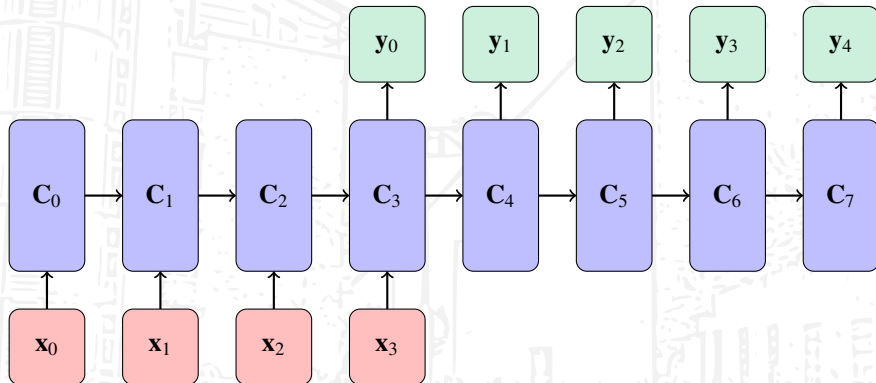
One to one, one to many, many to one

RNN come in many shapes: **one to one** (classical **feedforward neural network**), **one to many** (**image to caption**, „ein Bild sagt mehr als tausend Worte“), **many to one** (e.g., **sentiment analysis**, decision problems, classification of input of variable length/size):



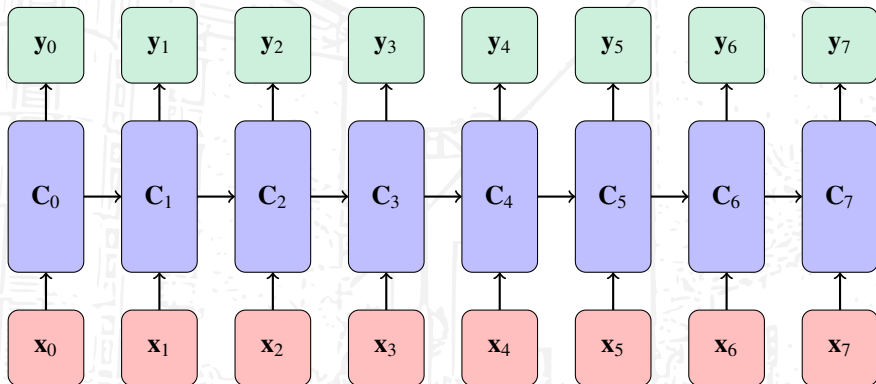
Many to many (first variant)

Many to many (delayed, e.g., **text translation**, texts in different languages are of different lengths: say “Hello!” to Siri, Alexa, Google translate):



Many to many (second variant)

Many to many (synced, e.g., classification of video frames; **detection of objects in autonomous driving**; automated video editing; medical image registration):



Simple RNN

Particular simple RNN are the so-called **Jordan** and **Elman** networks.

A **Jordan network** couples the hidden state with the previous **output** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hy}\mathbf{y}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0,$$

an **Elman network** couples the hidden state with the previous **hidden state** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

Initial output/state $\mathbf{y}_{-1} = \mathbf{h}_{-1} = \mathbf{o}$. Sizes of the vectors: $\mathbf{x}_t \in \mathbb{R}^{n_i}$, $\mathbf{h}_t \in \mathbb{R}^{n_h}$, $\mathbf{y}_t \in \mathbb{R}^{n_o}$.

Defining parameters:

- ▶ **weight matrices:** $\mathbf{W}_{hy} \in \mathbb{R}^{n_h \times n_o}$, $\mathbf{W}_{hh} \in \mathbb{R}^{n_h \times n_h}$, $\mathbf{W}_{hx} \in \mathbb{R}^{n_h \times n_i}$, and $\mathbf{W}_{yh} \in \mathbb{R}^{n_o \times n_h}$,
- ▶ **biases:** $\mathbf{b}_h \in \mathbb{R}^{n_h}$ and $\mathbf{b}_y \in \mathbb{R}^{n_o}$,
- ▶ **activation functions:** a_h for the hidden state (default: TanH), a_y for the output (e.g., SoftMax).

Backpropagation through time

RNN are trained by **unfolding** the RNN and using the **backpropagation algorithm**. This technique is known as **backpropagation through time (BPTT)**. We need the **derivatives of all states** for every time t for **finite sequences**, as we can only use those during training due to memory restrictions.

We consider an **Elman RNN**

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

unfolded for $k + 1$ steps, **two finite sequences** $\{\mathbf{x}_t\}_{t=0}^k$ and $\{\mathbf{y}_t\}_{t=0}^k$, and a **cost function**

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

To **simplify notation**, we define

$$\mathbf{a}_t := \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h, \quad \mathbf{z}_t := \mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y, \quad t \in \mathbb{N}_0.$$

Backpropagation through time

Theorem (BPTT for Elman RNN)

Let an Elman RNN unfolded for $k + 1$ steps, two finite sequences $\{\mathbf{x}_t\}_{t=0}^k$ and $\{\mathbf{y}_t\}_{t=0}^k$, and a cost function

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t)$$

be given. Then the derivatives with respect to the hidden states can be computed recursively starting at the end time $t = k$ and proceeding backwards through time, $0 \leq t < k$,

$$\frac{\partial C}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \left(a'_h(\mathbf{a}_{t+1}) \circ \frac{\partial C}{\partial \mathbf{h}_{t+1}} \right) + \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t} \right),$$

where in case $t = k$ the terms involving $t + 1$ are omitted.

Backpropagation through time

Theorem (BPTT for Elman RNN II)

The derivatives of the cost function with respect to the two biases, three weight matrices, and $k + 1$ inputs are given by

$$\frac{\partial C}{\partial \mathbf{b}_y} = \sum_{t=0}^k a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t},$$

$$\frac{\partial C}{\partial \mathbf{b}_h} = \sum_{t=0}^k a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t},$$

$$\frac{\partial C}{\partial \mathbf{W}_{yh}} = \sum_{t=0}^k \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t} \right) \mathbf{h}_t^\top,$$

$$\frac{\partial C}{\partial \mathbf{W}_{hh}} = \sum_{t=0}^k \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right) \mathbf{h}_{t-1}^\top,$$

$$\frac{\partial C}{\partial \mathbf{W}_{hx}} = \sum_{t=0}^k \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right) \mathbf{x}_t^\top,$$

$$\frac{\partial C}{\partial \mathbf{x}_t} = \mathbf{W}_{hx}^\top \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right), \quad t = 0, \dots, k.$$

Outline

Repetition

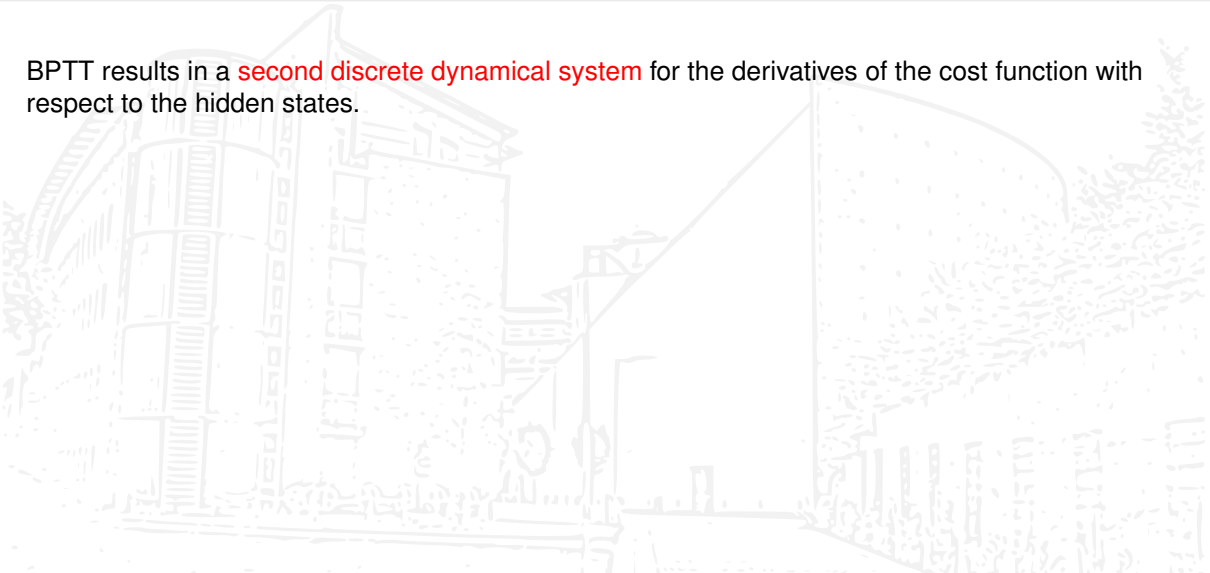
- Recurrent Neural Networks
- Cells & layers
- Elman network
- Backpropagation & Training

RNN: properties & variants

- Problems w/ vanilla RNN
- Long-term dependencies
- LSTM & variants
- BPTT for LSTM

Problems & simple solutions

BPTT results in a **second discrete dynamical system** for the derivatives of the cost function with respect to the hidden states.



Problems & simple solutions

BPTT results in a **second discrete dynamical system** for the derivatives of the cost function with respect to the hidden states.

The **main contribution** of the recurrence of the hidden states forward through time and of the derivatives backward through time lies in the **matrix multiplications** by $\mathbf{W}_{hh}$ and $\mathbf{W}_{hh}^T$.

Problems & simple solutions

BPTT results in a **second discrete dynamical system** for the derivatives of the cost function with respect to the hidden states.

The **main contribution** of the recurrence of the hidden states forward through time and of the derivatives backward through time lies in the **matrix multiplications** by $\mathbf{W}_{hh}$ and $\mathbf{W}_{hh}^T$.

If all eigenvalues of $\mathbf{W}_{hh}$ are less than one in absolute value, the **gradients will decay to zero**, if there exists an eigenvalue of absolute value larger than one, the **gradients will explode**.

Problems & simple solutions

BPTT results in a **second discrete dynamical system** for the derivatives of the cost function with respect to the hidden states.

The **main contribution** of the recurrence of the hidden states forward through time and of the derivatives backward through time lies in the **matrix multiplications** by $\mathbf{W}_{hh}$ and $\mathbf{W}_{hh}^T$.

If all eigenvalues of $\mathbf{W}_{hh}$ are less than one in absolute value, the **gradients will decay to zero**, if there exists an eigenvalue of absolute value larger than one, the **gradients will explode**.

To prevent this **problem of the vanishing/exploding gradients**, the matrix $\mathbf{W}_{hh}$ is initialized by a **random orthogonal matrix** and techniques like **gradient clipping** are used.

Problems & simple solutions

BPTT results in a **second discrete dynamical system** for the derivatives of the cost function with respect to the hidden states.

The **main contribution** of the recurrence of the hidden states forward through time and of the derivatives backward through time lies in the **matrix multiplications** by $\mathbf{W}_{hh}$ and $\mathbf{W}_{hh}^T$.

If all eigenvalues of $\mathbf{W}_{hh}$ are less than one in absolute value, the **gradients will decay to zero**, if there exists an eigenvalue of absolute value larger than one, the **gradients will explode**.

To prevent this **problem of the vanishing/exploding gradients**, the matrix $\mathbf{W}_{hh}$ is initialized by a **random orthogonal matrix** and techniques like **gradient clipping** are used.

The **activation functions** should not amplify, thus often the **hyperbolic tangent** is used, since $|\text{Tanh}(x)| \leq 1$ and $|\text{Tanh}'(x)| = |1 - \text{Tanh}^2(x)| \leq 1$ ($\text{Tanh}(0) = 0$).

Long-term dependencies

All these techniques can not circumvent one major problem: that of **long-term dependencies**.



Long-term dependencies

All these techniques can not circumvent one major problem: that of **long-term dependencies**.

As the hidden state is transformed in every step by a **matrix multiplication**, long-term dependencies are **hard to learn**. The classical example is a sentence like

*I was born in **France**. [...] I speak **French**.*

Long-term dependencies

All these techniques can not circumvent one major problem: that of **long-term dependencies**.

As the hidden state is transformed in every step by a **matrix multiplication**, long-term dependencies are **hard to learn**. The classical example is a sentence like

*I was born in **France**. [...] I speak **French**.*

The longer the part in brackets, the smaller the probability that the RNN learns this long-term dependency. RNN can only learn dependencies that are as long ago **as the unrolling parameter permits**. In many cases RNN do not learn dependencies that are too long-term; simple RNN only have a **short-term memory**.

Long-term dependencies

All these techniques can not circumvent one major problem: that of **long-term dependencies**.

As the hidden state is transformed in every step by a **matrix multiplication**, long-term dependencies are **hard to learn**. The classical example is a sentence like

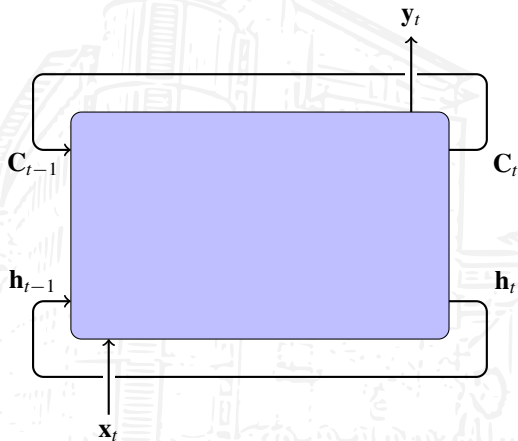
*I was born in **France**. [...] I speak **French**.*

The longer the part in brackets, the smaller the probability that the RNN learns this long-term dependency. RNN can only learn dependencies that are as long ago **as the unrolling parameter permits**. In many cases RNN do not learn dependencies that are too long-term; simple RNN only have a **short-term memory**.

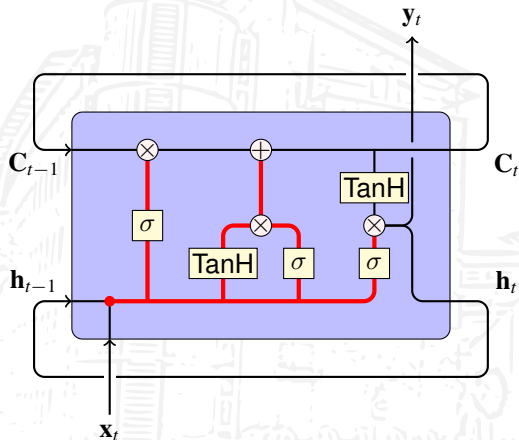
The solution to deal with long-term dependencies are **Long Short-Term Memory (LSTM)** networks. These have been published in Hochreiter and Schmidhuber [2]; the presentation here follows the blog by Olah [3].

LSTM

LSTM have a **hidden state $\mathbf{h}_t$** and an **additional cell state $\mathbf{C}_t$** that stores long-term dependencies.



LSTM

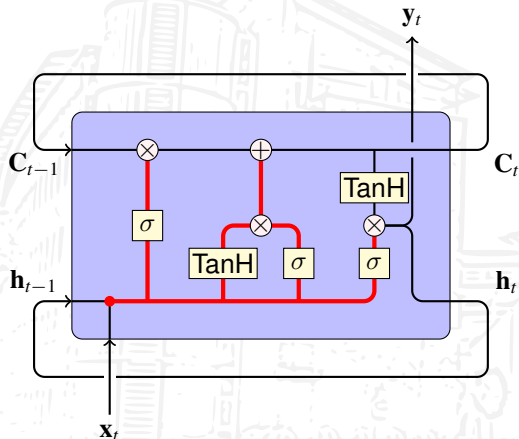


LSTM have a **hidden state h_t** and an **additional cell state C_t** that stores long-term dependencies.

The inner structure is **more complicated** than for a simple RNN, we have

- ▶ **five classical layers** w/ weight matrices, biases, thrice the logistic function σ , two activation functions, here twice TanH ,
- ▶ three **componentwise multiplications** and one **componentwise addition**,
- ▶ one **vector concatenation**, five **copies**.

LSTM



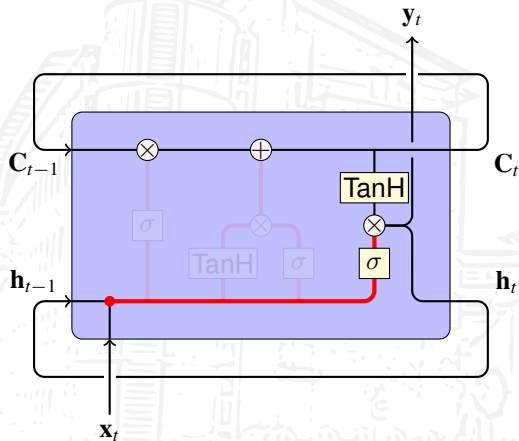
LSTM have a **hidden state h_t** and an **additional cell state C_t** that stores long-term dependencies.

The inner structure is **more complicated** than for a simple RNN, we have

- ▶ **five classical layers** w/ weight matrices, biases, thrice the logistic function σ , two activation functions, here twice TanH,
- ▶ three **componentwise multiplications** and one **componentwise addition**,
- ▶ one **vector concatenation**, five **copies**.

In this example the **output y_t** is equal to the **hidden state h_t** .

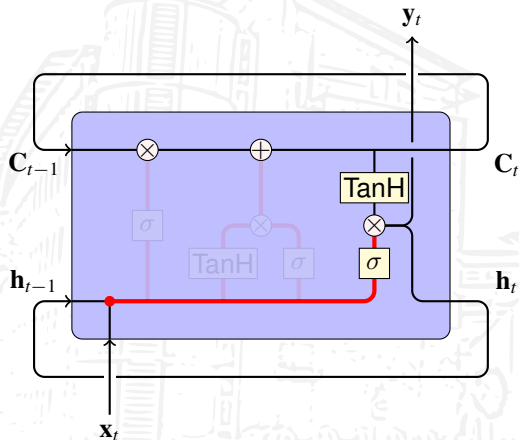
LSTM: update of the hidden state/output



In a **simple RNN** the **new hidden state** is computed from the **current input $\mathbf{x}_t$** and the **old hidden state $\mathbf{h}_{t-1}$** ,

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h).$$

LSTM: update of the hidden state/output



In a **simple RNN** the **new hidden state** is computed from the **current input $\mathbf{x}_t$** and the **old hidden state $\mathbf{h}_{t-1}$** ,

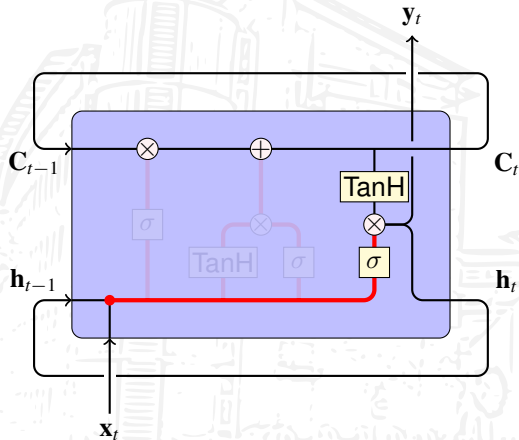
$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h).$$

In an **LSTM** the **new hidden state** is computed from the **current input $\mathbf{x}_t$** , the **old hidden state $\mathbf{h}_{t-1}$** , and the **current cell state $\mathbf{C}_t$** :

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right),$$

$$\text{where } \mathbf{W}_h := \begin{pmatrix} \mathbf{W}_{hh} & \mathbf{W}_{hx} \end{pmatrix}.$$

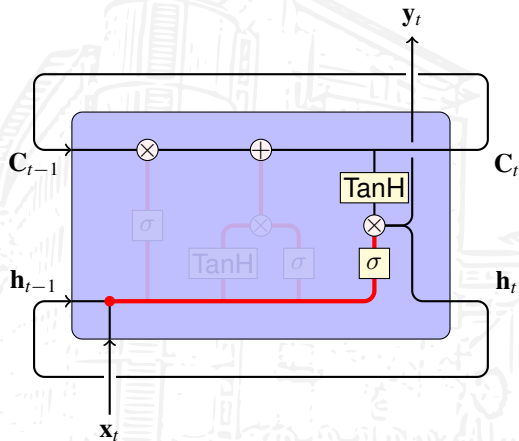
LSTM: update of the hidden state/output



The output of the logistic function acts as an **output gate** that decides which parts of the current cell state C_t activated by TanH are the next hidden state:

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right).$$

LSTM: update of the hidden state/output

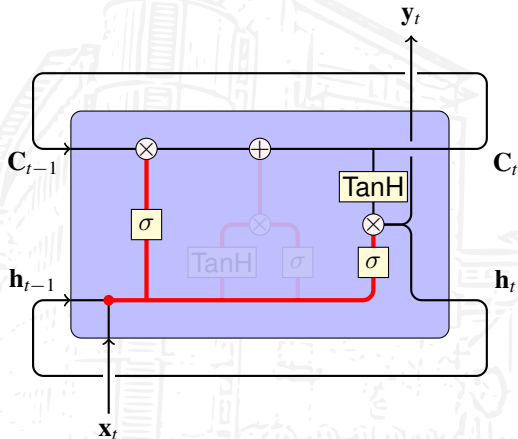


The output of the logistic function acts as an **output gate** that decides which parts of the current cell state C_t activated by TanH are the next hidden state:

$$\mathbf{h}_t = \text{TanH}(C_t) \circ \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right).$$

In a **language example**, if the last word was a **subject**, we might output whether the subject was **singular** or **plural** to **conjugate a verb** accordingly.

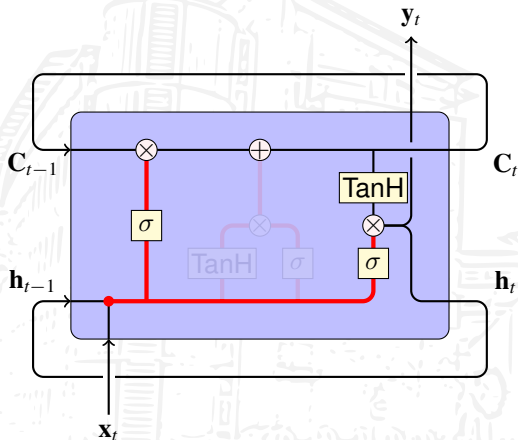
LSTM: forget gate



The **cell state is modified** using

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right)$$

LSTM: forget gate



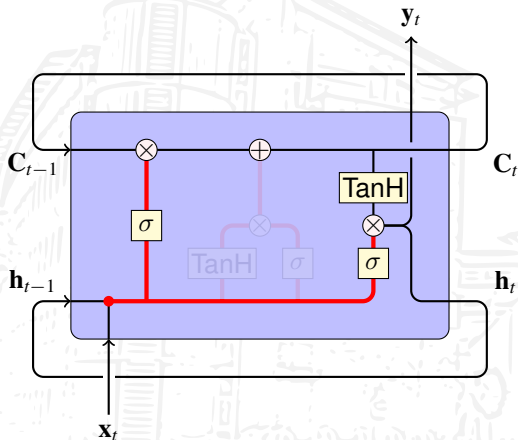
The **cell state is modified** using

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right)$$

that is **multiplied componentwise** to the old cell state $\mathbf{C}_{t-1}$,

$$\hat{\mathbf{C}}_{t-1} = \mathbf{C}_{t-1} \circ \mathbf{f}_t.$$

LSTM: forget gate



The **cell state is modified** using

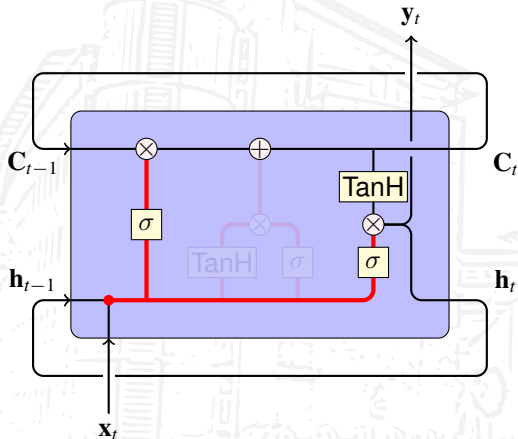
$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right)$$

that is **multiplied componentwise** to the old cell state $\mathbf{C}_{t-1}$,

$$\hat{\mathbf{C}}_{t-1} = \mathbf{C}_{t-1} \circ \mathbf{f}_t.$$

For every component in $\mathbf{C}_{t-1}$ a **number in $(0, 1)$** is computed based on the last hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$.

LSTM: forget gate

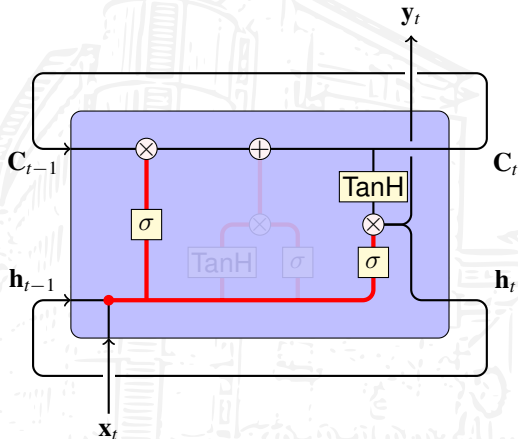


The **cell state is modified** using

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

$$\hat{\mathbf{C}}_{t-1} = \mathbf{C}_{t-1} \circ \mathbf{f}_t.$$

LSTM: forget gate



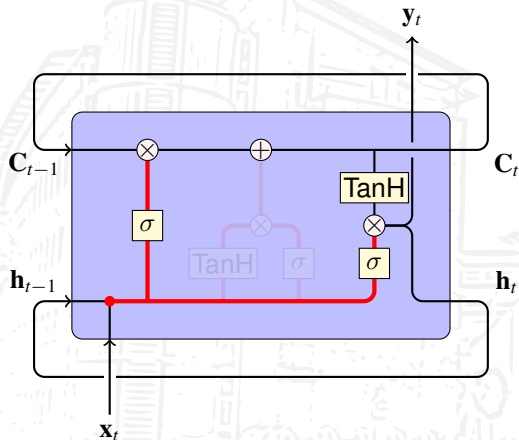
The **cell state** is modified using

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

$$\hat{\mathbf{C}}_{t-1} = \mathbf{C}_{t-1} \circ \mathbf{f}_t.$$

A **zero** in $\mathbf{f}_t$ means **forget that part completely**, a **one** means **keep that part completely**.

LSTM: forget gate



The **cell state** is **modified** using

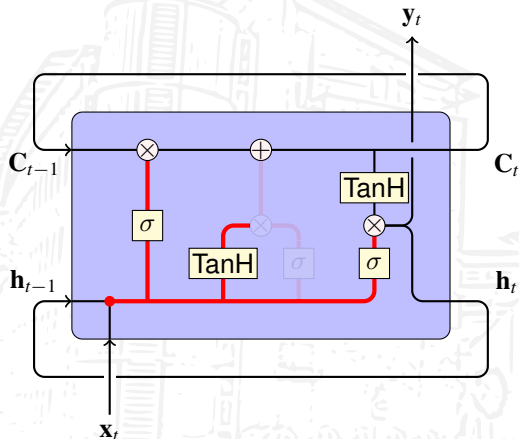
$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

$$\hat{\mathbf{C}}_{t-1} = \mathbf{C}_{t-1} \circ \mathbf{f}_t.$$

A **zero** in $\mathbf{f}_t$ means **forget that part completely**, a **one** means **keep that part completely**.

In the **language example**, we forget about the **gender** of the old subject when a new subject with a **different gender** occurs.

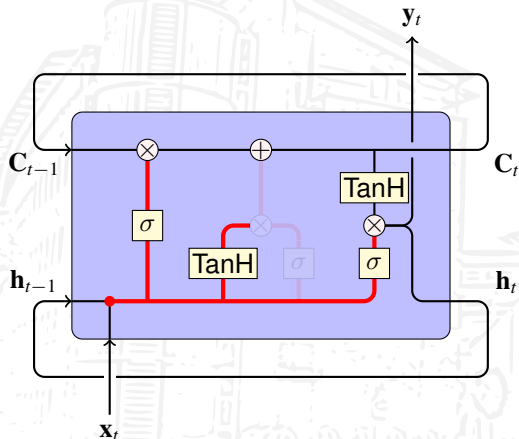
LSTM: candidate for the next cell state



A candidate for the next cell state is computed as

$$\tilde{c}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right).$$

LSTM: candidate for the next cell state

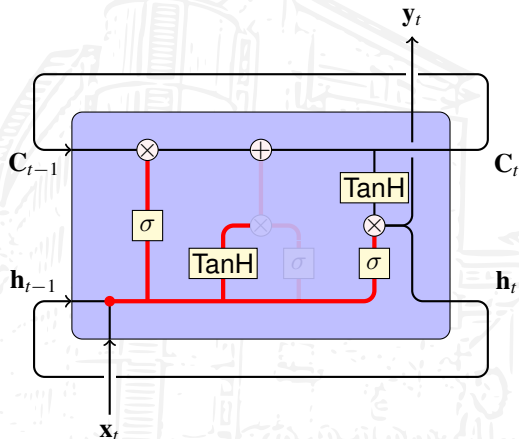


A candidate for the next cell state is computed as

$$\tilde{c}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right).$$

In the **language model**, here the information of the **gender of the new subject** might enter.

LSTM: candidate for the next cell state



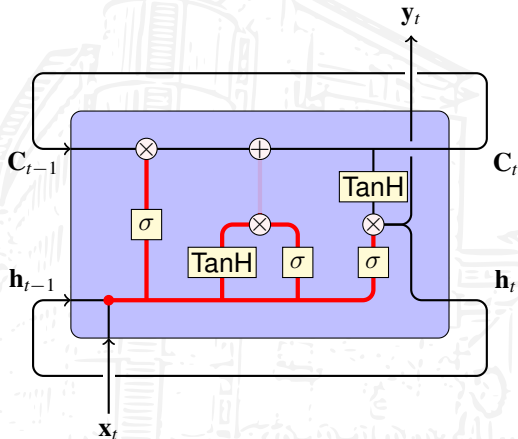
A **candidate for the next cell state** is computed as

$$\tilde{C}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right).$$

In the **language model**, here the information of the **gender of the new subject** might enter.

To compute the new cell state, we have to decide how to **mix** $\hat{C}_{t-1} = C_{t-1} \circ \mathbf{f}_t$, the **old cell state** changed by $\mathbf{f}_t$, with the **candidate cell state** $\tilde{C}_t$.

LSTM: input gate



Which parts of the candidate cell state

$$\tilde{\mathbf{C}}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right)$$

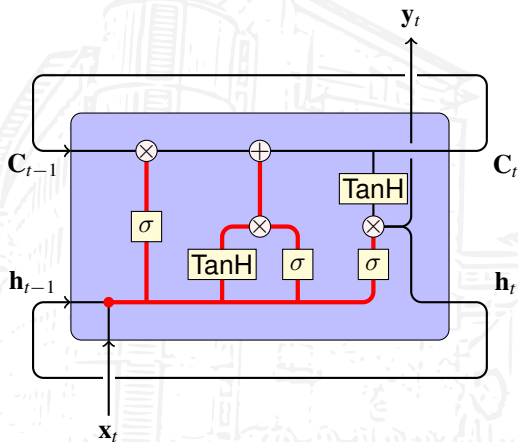
enter the scene is determined by

$$\mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right)$$

and componentwise multiplication,

$$\check{\mathbf{C}}_t = \tilde{\mathbf{C}}_t \circ \mathbf{i}_t.$$

LSTM: cell state update

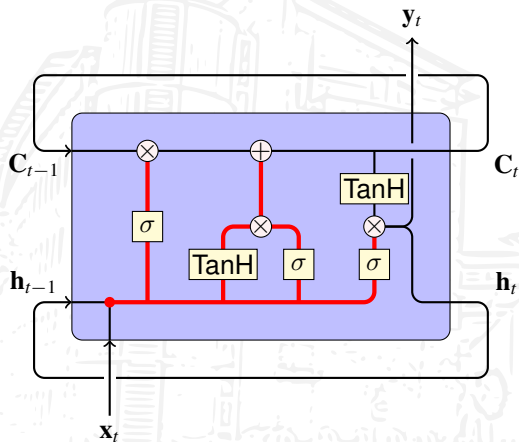


The **new cell state** is determined as the sum of

- ▶ the **old cell state** modified by the **forget gate** and
- ▶ the **new cell state candidate** modified by the **input gate**,

$$\begin{aligned} C_t &= \hat{C}_{t-1} + \check{C}_t \\ &= C_{t-1} \circ f_t + \tilde{C}_t \circ i_t. \end{aligned}$$

LSTM: cell state update



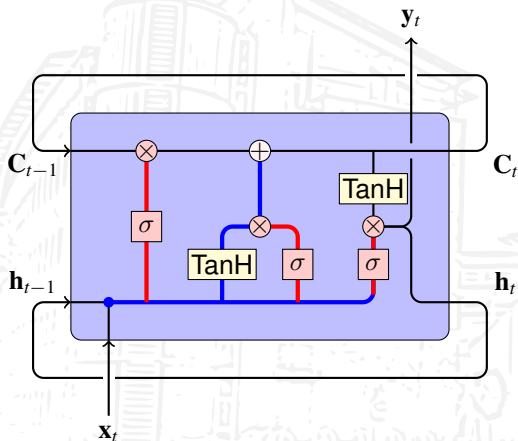
The **new cell state** is determined as the sum of

- ▶ the **old cell state** modified by the **forget gate** and
- ▶ the **new cell state candidate** modified by the **input gate**,

$$\begin{aligned} c_t &= \hat{c}_{t-1} + \check{c}_t \\ &= c_{t-1} \circ f_t + \tilde{c}_t \circ i_t. \end{aligned}$$

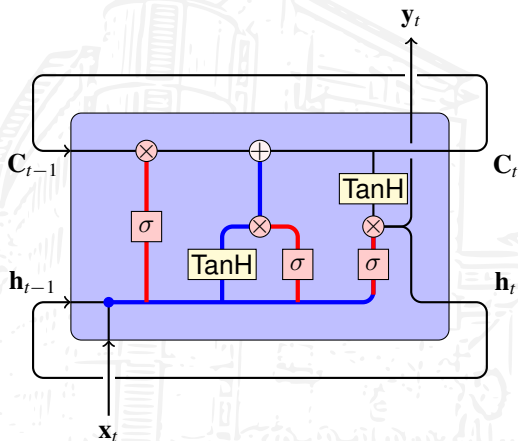
The **notation** can be memorized: $\hat{c}$ denotes that quantities are **moved out of the cell state**, $\check{c}$ is used to denote that quantities are **moved into the cell state**.

LSTM: input, output, and forget gate



The **componentwise multiplications** with the results of the **logistic functions** are termed **gates**.

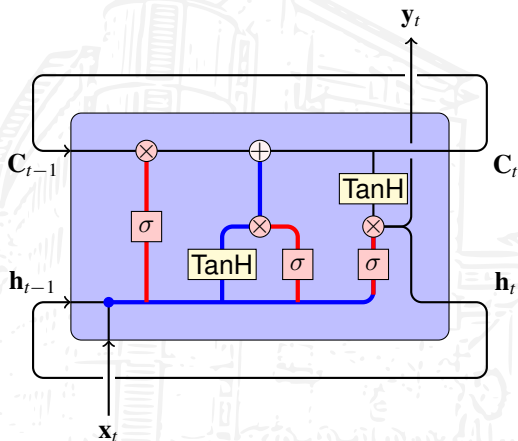
LSTM: input, output, and forget gate



The **componentwise multiplications** with the results of the **logistic functions** are termed **gates**.

An LSTM has **three gates**:

LSTM: input, output, and forget gate

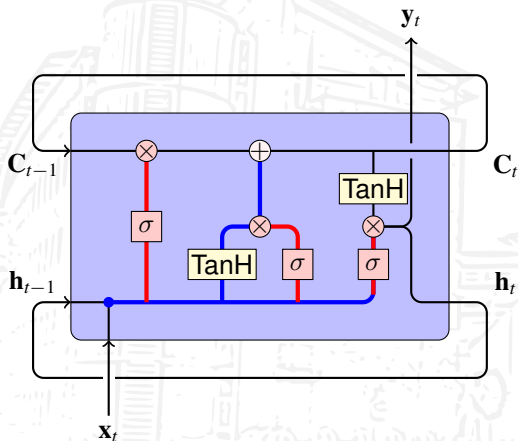


The **componentwise multiplications** with the results of the **logistic functions** are termed **gates**.

An LSTM has **three gates**: a **forget gate**,

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

LSTM: input, output, and forget gate



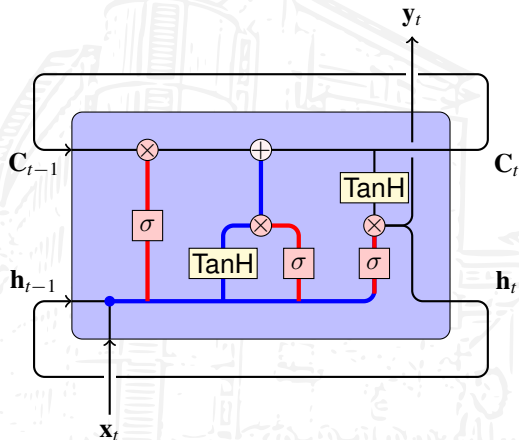
The **componentwise multiplications** with the results of the **logistic functions** are termed **gates**.

An LSTM has **three gates**: a **forget gate**, an **input gate**,

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

$$\mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right),$$

LSTM: input, output, and forget gate



The **componentwise multiplications** with the results of the **logistic functions** are termed **gates**.

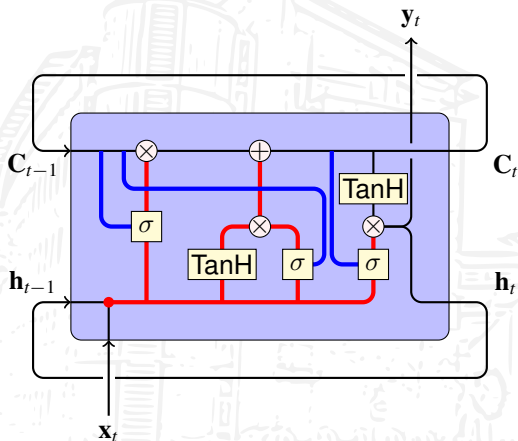
An LSTM has **three gates**: a **forget gate**, an **input gate**, and an **output gate**,

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

$$\mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right),$$

$$\mathbf{m}_t = \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right).$$

LSTM variants: peephole connections



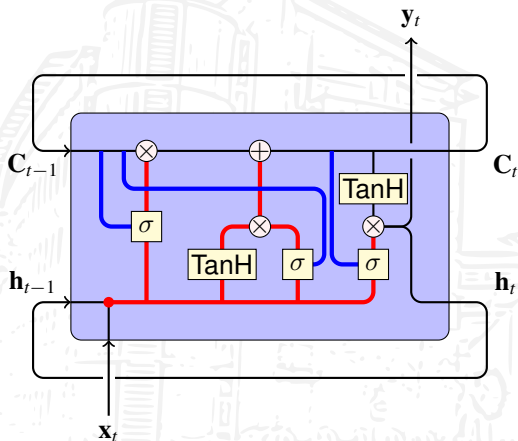
There exist **variants** that introduce **peephole** connections that **allow the gates to see the cell state**, e.g., the forget gate looks as follows:

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{C}_{t-1} \\ \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

where $\mathbf{W}_f$ splits into

$$\mathbf{W}_f = (\mathbf{W}_{fc} \quad \mathbf{W}_{fh} \quad \mathbf{W}_{fx}).$$

LSTM variants: peephole connections



There exist **variants** that introduce **peephole**s that **allow the gates to see the cell state**, e.g., the forget gate looks as follows:

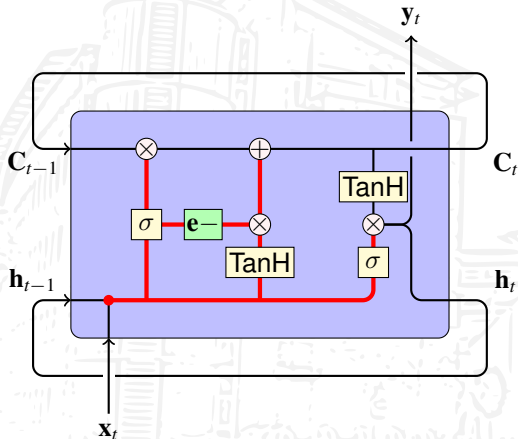
$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{C}_{t-1} \\ \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right),$$

where $\mathbf{W}_f$ splits into

$$\mathbf{W}_f = (\mathbf{W}_{fc} \quad \mathbf{W}_{fh} \quad \mathbf{W}_{fx}).$$

This is a **generalization**: In case $\mathbf{W}_{fc} = \mathbf{O}$ we **recover the variant w/o peephole**.

LSTM variants: coupled input and forget gates

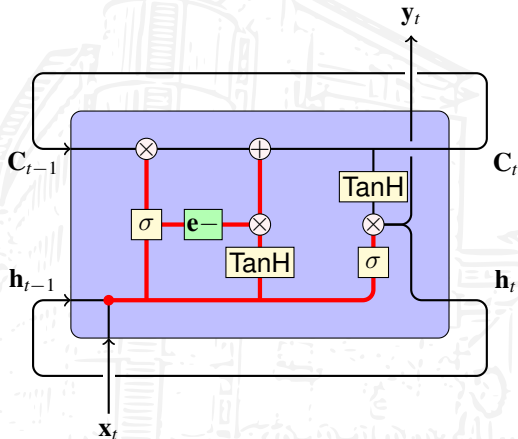


Using a model with **limited memory**, we insert something new when we forget something. Thus, a **variation on LSTM** is to **couple the input and forget gates**,

$$C_t = C_{t-1} \circ f_t + \tilde{C}_t \circ (e - f_t),$$

where e is the vector of all ones.

LSTM variants: coupled input and forget gates



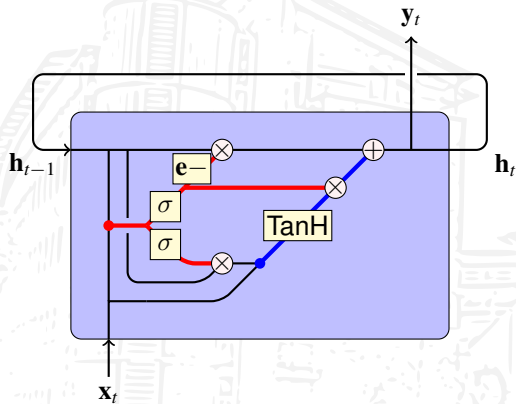
Using a model with **limited memory**, we insert something new when we forget something. Thus, a **variation on LSTM** is to **couple the input and forget gates**,

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ (\mathbf{e} - \mathbf{f}_t),$$

where $\mathbf{e}$ is the vector of all ones.

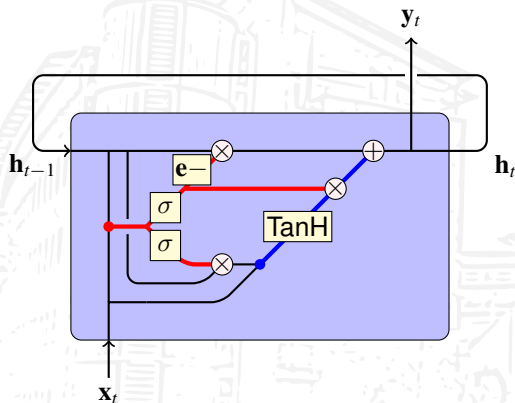
Here, $\mathbf{i}_t = \mathbf{e} - \mathbf{f}_t$ and we compute the new cell state as a **convex combination** of the old cell state and the new candidate cell state.

LSTM variants: GRU



The **Gated Recurrent Unit (GRU)** by Cho et al. [1] is a **more dramatic variation of an LSTM**.

LSTM variants: GRU



The **Gated Recurrent Unit (GRU)** by Cho et al. [1] is a **more dramatic variation of an LSTM**.

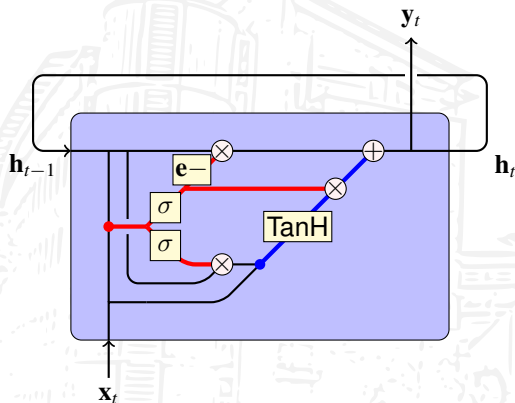
Here, **cell and hidden state are merged**, **input and forget gate are coupled**, and **no bias is used**,

$$\mathbf{z}_t = \sigma \left(\mathbf{W}_z \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} \right), \quad \mathbf{r}_t = \sigma \left(\mathbf{W}_r \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} \right),$$

$$\tilde{\mathbf{h}}_t = \text{TanH} \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \circ \mathbf{r}_t \\ \mathbf{x}_t \end{pmatrix} \right),$$

$$\mathbf{h}_t = (\mathbf{e} - \mathbf{z}_t) \circ \mathbf{h}_{t-1} + \mathbf{z}_t \circ \tilde{\mathbf{h}}_t.$$

LSTM variants: GRU



The **Gated Recurrent Unit (GRU)** by Cho et al. [1] is a **more dramatic variation of an LSTM**.

Here, **cell and hidden state are merged**, **input and forget gate are coupled**, and **no bias is used**,

$$\mathbf{z}_t = \sigma \left(\mathbf{W}_z \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} \right), \quad \mathbf{r}_t = \sigma \left(\mathbf{W}_r \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} \right),$$

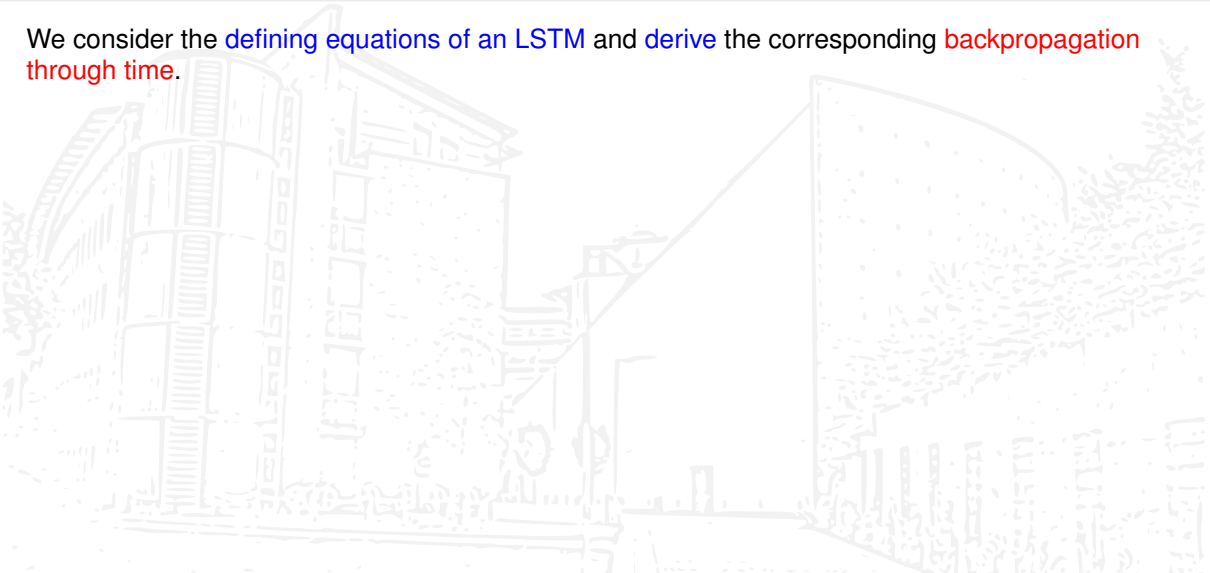
$$\tilde{\mathbf{h}}_t = \text{TanH} \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \circ \mathbf{r}_t \\ \mathbf{x}_t \end{pmatrix} \right),$$

$$\mathbf{h}_t = (\mathbf{e} - \mathbf{z}_t) \circ \mathbf{h}_{t-1} + \mathbf{z}_t \circ \tilde{\mathbf{h}}_t.$$

Only **two gates**: **update gate $\mathbf{z}_t$** & **reset gate $\mathbf{r}_t$** .

BPTT for LSTM

We consider the defining equations of an LSTM and derive the corresponding backpropagation through time.



BPTT for LSTM

We consider the **defining equations of an LSTM** and **derive** the corresponding **backpropagation through time**.

The LSTM is based on the **three gates** (forget gate, input gate, output gate),

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right), \quad \mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right), \quad \mathbf{m}_t = \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right),$$

BPTT for LSTM

We consider the **defining equations of an LSTM** and **derive** the corresponding **backpropagation through time**.

The LSTM is based on the **three gates** (forget gate, input gate, output gate),

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right), \quad \mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right), \quad \mathbf{m}_t = \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right),$$

the **cell state update**

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \tilde{\mathbf{C}}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right),$$

BPTT for LSTM

We consider the **defining equations of an LSTM** and **derive** the corresponding **backpropagation through time**.

The LSTM is based on the **three gates** (forget gate, input gate, output gate),

$$\mathbf{f}_t = \sigma \left(\mathbf{W}_f \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f \right), \quad \mathbf{i}_t = \sigma \left(\mathbf{W}_i \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_i \right), \quad \mathbf{m}_t = \sigma \left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_h \right),$$

the **cell state update**

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \tilde{\mathbf{C}}_t = \text{TanH} \left(\mathbf{W}_C \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_C \right),$$

and the **hidden state update** that depends on the previous hidden state,

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

BPTT for LSTM

Like in the simple RNN case we consider a **cost function**, here $\mathbf{y}_t = \mathbf{h}_t$,

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

BPTT for LSTM

Like in the simple RNN case we consider a **cost function**, here $\mathbf{y}_t = \mathbf{h}_t$,

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

The **hidden state evolves** as follows,

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t = f(\mathbf{h}_{t-1}).$$

BPTT for LSTM

Like in the simple RNN case we consider a **cost function**, here $\mathbf{y}_t = \mathbf{h}_t$,

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

The **hidden state evolves** as follows,

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t = f(\mathbf{h}_{t-1}).$$

For the last **hidden state in step $t = k$** we can easily compute

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k},$$

BPTT for LSTM

Like in the simple RNN case we consider a **cost function**, here $\mathbf{y}_t = \mathbf{h}_t$,

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

The **hidden state evolves** as follows,

$$\mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t = f(\mathbf{h}_{t-1}).$$

For the last **hidden state in step** $t = k$ we can easily compute

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k},$$

for the **previous steps** $t < k$ we end up with the **recursion**

$$\frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial C_t}{\partial \mathbf{h}_t} + \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \cdot \frac{\partial C}{\partial \mathbf{h}_{t+1}}.$$

BPTT for LSTM

A **similar recursion** develops for the **cell state $\mathbf{C}_t$** , which occurs in the two formulas

$$\mathbf{C}_{t+1} = \mathbf{C}_t \circ \mathbf{f}_{t+1} + \tilde{\mathbf{C}}_{t+1} \circ \mathbf{i}_{t+1}, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

BPTT for LSTM

A **similar recursion** develops for the **cell state $\mathbf{C}_t$** , which occurs in the two formulas

$$\mathbf{C}_{t+1} = \mathbf{C}_t \circ \mathbf{f}_{t+1} + \tilde{\mathbf{C}}_{t+1} \circ \mathbf{i}_{t+1}, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

In the **last step $t = k$** we only have to compute

$$\frac{\partial \mathcal{C}}{\partial \mathbf{C}_k} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{C}_k} \cdot \frac{\partial \mathcal{C}}{\partial \mathbf{h}_k} = \left(\mathbf{e} - \text{TanH}^2(\mathbf{C}_k) \right) \circ \mathbf{m}_k \circ \frac{\partial \mathcal{C}}{\partial \mathbf{h}_k},$$

BPTT for LSTM

A **similar recursion** develops for the **cell state $\mathbf{C}_t$** , which occurs in the two formulas

$$\mathbf{C}_{t+1} = \mathbf{C}_t \circ \mathbf{f}_{t+1} + \tilde{\mathbf{C}}_{t+1} \circ \mathbf{i}_{t+1}, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

In the **last step $t = k$** we only have to compute

$$\frac{\partial C}{\partial \mathbf{C}_k} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{C}_k} \cdot \frac{\partial C}{\partial \mathbf{h}_k} = \left(\mathbf{e} - \text{TanH}^2(\mathbf{C}_k) \right) \circ \mathbf{m}_k \circ \frac{\partial C}{\partial \mathbf{h}_k},$$

in **previous steps $t < k$** we have the **recurrence**

$$\frac{\partial C}{\partial \mathbf{C}_t} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{C}_t} \cdot \frac{\partial C}{\partial \mathbf{h}_t} + \frac{\partial \mathbf{C}_{t+1}}{\partial \mathbf{C}_t} \frac{\partial C}{\partial \mathbf{C}_{t+1}} = \left(\mathbf{e} - \text{TanH}^2(\mathbf{C}_t) \right) \circ \mathbf{m}_t \circ \frac{\partial C}{\partial \mathbf{h}_t} + \mathbf{f}_{t+1} \circ \frac{\partial C}{\partial \mathbf{C}_{t+1}}.$$

BPTT for LSTM

A **similar recursion** develops for the **cell state $\mathbf{C}_t$** , which occurs in the two formulas

$$\mathbf{C}_{t+1} = \mathbf{C}_t \circ \mathbf{f}_{t+1} + \tilde{\mathbf{C}}_{t+1} \circ \mathbf{i}_{t+1}, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

In the **last step $t = k$** we only have to compute

$$\frac{\partial C}{\partial \mathbf{C}_k} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{C}_k} \cdot \frac{\partial C}{\partial \mathbf{h}_k} = \left(\mathbf{e} - \text{TanH}^2(\mathbf{C}_k) \right) \circ \mathbf{m}_k \circ \frac{\partial C}{\partial \mathbf{h}_k},$$

in **previous steps $t < k$** we have the **recurrence**

$$\frac{\partial C}{\partial \mathbf{C}_t} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{C}_t} \cdot \frac{\partial C}{\partial \mathbf{h}_t} + \frac{\partial \mathbf{C}_{t+1}}{\partial \mathbf{C}_t} \frac{\partial C}{\partial \mathbf{C}_{t+1}} = \left(\mathbf{e} - \text{TanH}^2(\mathbf{C}_t) \right) \circ \mathbf{m}_t \circ \frac{\partial C}{\partial \mathbf{h}_t} + \mathbf{f}_{t+1} \circ \frac{\partial C}{\partial \mathbf{C}_{t+1}}.$$

Thus, in an LSTM, we have **two recurrences**, one for the **hidden state**, one for the **cell state**.

BPTT for LSTM

We define

$$\mathbf{a}_t := \begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{m}_t \\ \tilde{\mathbf{C}}_t \end{pmatrix}, \quad \mathbf{z}_t := \begin{pmatrix} \mathbf{z}_{ft} \\ \mathbf{z}_{it} \\ \mathbf{z}_{ht} \\ \mathbf{z}_{Ct} \end{pmatrix} := \begin{pmatrix} \mathbf{W}_{fh} \mathbf{h}_{t-1} + \mathbf{W}_{fx} \mathbf{x}_t + \mathbf{b}_f \\ \mathbf{W}_{ih} \mathbf{h}_{t-1} + \mathbf{W}_{ix} \mathbf{x}_t + \mathbf{b}_i \\ \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{b}_h \\ \mathbf{W}_{Ch} \mathbf{h}_{t-1} + \mathbf{W}_{Cx} \mathbf{x}_t + \mathbf{b}_C \end{pmatrix} =: \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}.$$

BPTT for LSTM

We define

$$\mathbf{a}_t := \begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{m}_t \\ \tilde{\mathbf{C}}_t \end{pmatrix}, \quad \mathbf{z}_t := \begin{pmatrix} \mathbf{z}_{ft} \\ \mathbf{z}_{it} \\ \mathbf{z}_{ht} \\ \mathbf{z}_{Ct} \end{pmatrix} := \begin{pmatrix} \mathbf{W}_{fh} \mathbf{h}_{t-1} + \mathbf{W}_{fx} \mathbf{x}_t + \mathbf{b}_f \\ \mathbf{W}_{ih} \mathbf{h}_{t-1} + \mathbf{W}_{ix} \mathbf{x}_t + \mathbf{b}_i \\ \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{b}_h \\ \mathbf{W}_{Ch} \mathbf{h}_{t-1} + \mathbf{W}_{Cx} \mathbf{x}_t + \mathbf{b}_C \end{pmatrix} =: \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}.$$

We have (thrice [logistic function](#), once [hyperbolic tangent](#))

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{z}_t} = \text{diag} \begin{pmatrix} (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \\ (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \\ (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \\ \mathbf{e} - \tilde{\mathbf{C}}_t \circ \tilde{\mathbf{C}}_t \end{pmatrix}, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_h^\top, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} = \mathbf{W}_x^\top.$$

BPTT for LSTM

We define

$$\mathbf{a}_t := \begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{m}_t \\ \tilde{\mathbf{C}}_t \end{pmatrix}, \quad \mathbf{z}_t := \begin{pmatrix} \mathbf{z}_{ft} \\ \mathbf{z}_{it} \\ \mathbf{z}_{ht} \\ \mathbf{z}_{Ct} \end{pmatrix} := \begin{pmatrix} \mathbf{W}_{fh} \mathbf{h}_{t-1} + \mathbf{W}_{fx} \mathbf{x}_t + \mathbf{b}_f \\ \mathbf{W}_{ih} \mathbf{h}_{t-1} + \mathbf{W}_{ix} \mathbf{x}_t + \mathbf{b}_i \\ \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{b}_h \\ \mathbf{W}_{Ch} \mathbf{h}_{t-1} + \mathbf{W}_{Cx} \mathbf{x}_t + \mathbf{b}_C \end{pmatrix} =: \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}.$$

We have (thrice [logistic function](#), once [hyperbolic tangent](#))

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{z}_t} = \text{diag} \begin{pmatrix} (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \\ (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \\ (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \\ \mathbf{e} - \tilde{\mathbf{C}}_t \circ \tilde{\mathbf{C}}_t \end{pmatrix}, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_h^\top, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} = \mathbf{W}_x^\top.$$

On the next slide we consider the three gates and the candidate state by looking at the [partial derivatives with respect to \$\mathbf{z}_{ft}\$, \$\mathbf{z}_{it}\$, \$\mathbf{z}_{ht}\$, and \$\mathbf{z}_{Ct}\$](#) .

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the **derivatives** with respect to the **gates and candidate cell state**,

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the **derivatives** with respect to the **gates and candidate cell state**,

$$\frac{\partial C}{\partial \mathbf{z}_{ft}} = \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_{ft}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{f}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \circ \mathbf{C}_{t-1} \circ \frac{\partial C}{\partial \mathbf{C}_t},$$

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the **derivatives** with respect to the **gates and candidate cell state**,

$$\frac{\partial C}{\partial \mathbf{z}_{ft}} = \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_{ft}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{f}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \circ \mathbf{C}_{t-1} \circ \frac{\partial C}{\partial \mathbf{C}_t},$$

$$\frac{\partial C}{\partial \mathbf{z}_{it}} = \frac{\partial \mathbf{i}_t}{\partial \mathbf{z}_{it}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{i}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \circ \tilde{\mathbf{C}}_t \circ \frac{\partial C}{\partial \mathbf{C}_t},$$

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the **derivatives** with respect to the **gates and candidate cell state**,

$$\frac{\partial C}{\partial \mathbf{z}_{ft}} = \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_{ft}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{f}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \circ \mathbf{C}_{t-1} \circ \frac{\partial C}{\partial \mathbf{C}_t},$$

$$\frac{\partial C}{\partial \mathbf{z}_{it}} = \frac{\partial \mathbf{i}_t}{\partial \mathbf{z}_{it}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{i}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \circ \tilde{\mathbf{C}}_t \circ \frac{\partial C}{\partial \mathbf{C}_t},$$

$$\frac{\partial C}{\partial \mathbf{z}_{ht}} = \frac{\partial \mathbf{m}_t}{\partial \mathbf{z}_{ht}} \frac{\partial \mathbf{h}_t}{\partial \mathbf{m}_t} \frac{\partial C}{\partial \mathbf{h}_t} = (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \circ \text{TanH}(\mathbf{C}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t},$$

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \tilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \text{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the **derivatives** with respect to the **gates and candidate cell state**,

$$\begin{aligned} \frac{\partial C}{\partial \mathbf{z}_{ft}} &= \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_{ft}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{f}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \circ \mathbf{C}_{t-1} \circ \frac{\partial C}{\partial \mathbf{C}_t}, \\ \frac{\partial C}{\partial \mathbf{z}_{it}} &= \frac{\partial \mathbf{i}_t}{\partial \mathbf{z}_{it}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{i}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \circ \tilde{\mathbf{C}}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}, \\ \frac{\partial C}{\partial \mathbf{z}_{ht}} &= \frac{\partial \mathbf{m}_t}{\partial \mathbf{z}_{ht}} \frac{\partial \mathbf{h}_t}{\partial \mathbf{m}_t} \frac{\partial C}{\partial \mathbf{h}_t} = (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \circ \text{TanH}(\mathbf{C}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t}, \\ \frac{\partial C}{\partial \mathbf{z}_{Ct}} &= \frac{\partial \tilde{\mathbf{C}}_t}{\partial \mathbf{z}_{Ct}} \frac{\partial \mathbf{C}_t}{\partial \tilde{\mathbf{C}}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \tilde{\mathbf{C}}_t \circ \tilde{\mathbf{C}}_t) \circ \mathbf{i}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}. \end{aligned}$$

BPTT for LSTM

Now we can compute the **missing part** that will be **handed to the previous time step**,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\top \frac{\partial C}{\partial \mathbf{z}_t},$$

BPTT for LSTM

Now we can compute the **missing part** that will be **handed to the previous time step**,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\top \frac{\partial C}{\partial \mathbf{z}_t},$$

and the **change in the input** (**handed to the previous layer**)

$$\frac{\partial C}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_x^\top \frac{\partial C}{\partial \mathbf{z}_t}.$$

BPTT for LSTM

Now we can compute the **missing part** that will be **handed to the previous time step**,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\top \frac{\partial C}{\partial \mathbf{z}_t},$$

and the **change in the input** (**handed to the previous layer**)

$$\frac{\partial C}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_x^\top \frac{\partial C}{\partial \mathbf{z}_t}.$$

The **update of weights and biases** (shared over time) is given by

$$\frac{\partial C}{\partial \mathbf{W}_h} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{h}_{t-1}^\top,$$

BPTT for LSTM

Now we can compute the **missing part** that will be **handed to the previous time step**,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\top \frac{\partial C}{\partial \mathbf{z}_t},$$

and the **change in the input** (**handed to the previous layer**)

$$\frac{\partial C}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_x^\top \frac{\partial C}{\partial \mathbf{z}_t}.$$

The **update of weights and biases** (shared over time) is given by

$$\frac{\partial C}{\partial \mathbf{W}_h} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{h}_{t-1}^\top, \quad \frac{\partial C}{\partial \mathbf{W}_x} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{x}_t^\top,$$

BPTT for LSTM

Now we can compute the **missing part** that will be **handed to the previous time step**,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\top \frac{\partial C}{\partial \mathbf{z}_t},$$

and the **change in the input** (**handed to the previous layer**)

$$\frac{\partial C}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_x^\top \frac{\partial C}{\partial \mathbf{z}_t}.$$

The **update of weights and biases** (shared over time) is given by

$$\frac{\partial C}{\partial \mathbf{W}_h} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{h}_{t-1}^\top, \quad \frac{\partial C}{\partial \mathbf{W}_x} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{x}_t^\top, \quad \frac{\partial C}{\partial \mathbf{b}} = \sum_{t=0}^k \frac{\partial C}{\partial \mathbf{z}_t}.$$

BPTT for LSTM

The implementation of BPTT for LSTM is based on **transferring the state** given by

$$\mathbf{W}_h^T \frac{\partial C}{\partial \mathbf{z}_t}, \quad \mathbf{f}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}$$

to the **previous step** and **computing all partial derivatives for the current time step**, updating the **derivatives** of the weights and biases.

BPTT for LSTM

The implementation of BPTT for LSTM is based on **transferring the state** given by

$$\mathbf{W}_h^T \frac{\partial C}{\partial \mathbf{z}_t}, \quad \mathbf{f}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}$$

to the **previous step** and **computing all partial derivatives for the current time step**, updating the **derivatives** of the weights and biases.

As optimizers all variants of **SGD** can be used, e.g., **Adam**.

BPTT for LSTM

The implementation of BPTT for LSTM is based on **transferring the state** given by

$$\mathbf{W}_h^T \frac{\partial C}{\partial \mathbf{z}_t}, \quad \mathbf{f}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}$$

to the **previous step** and **computing all partial derivatives for the current time step**, updating the **derivatives** of the weights and biases.

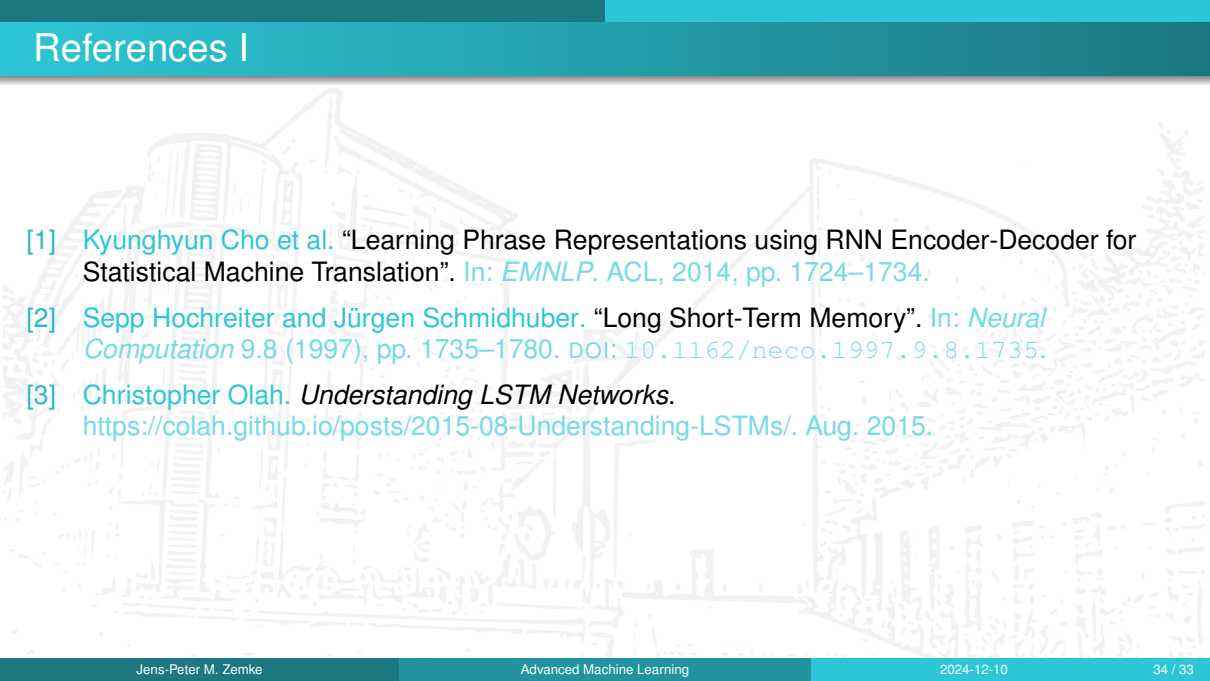
As optimizers all variants of **SGD** can be used, e.g., **Adam**.

A list of examples that show results obtained using LSTM can be found at **Andrej Karpathy's Blog**:

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

(You'll recognize many things you saw in this and the last lecture.)

References I

- 
- [1] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: *EMNLP. ACL*, 2014, pp. 1724–1734.
 - [2] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
 - [3] Christopher Olah. *Understanding LSTM Networks*. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Aug. 2015.